# AgileFlow AI

## Backend Implementation Guide

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

**Document Purpose**

This guide defines how the FastAPI backend of AgileFlow AI should be structured, implemented, tested, documented, and integrated with all agent components. It is written for the implementation foundation team and all component developers.

| Section | Purpose |
|---|---|
| Backend structure | Standard folders, modules, files, and layer responsibilities. |
| API rules | Consistent REST endpoints, request/response patterns, and error responses. |
| Database rules | MongoDB collection usage, IDs, schema consistency, and traceability. |
| Agent integration | How each agent reads/writes through the Shared Knowledge Base. |
| Quality rules | Testing, validation, documentation, and AI-agent-safe development practices. |

*Prepared for AgileFlow AI implementation environment setup*

# 1. Backend Implementation Goal

The backend is the control and data layer of AgileFlow AI. It must support the complete human-in-the-loop Agile SDLC workflow, agent handovers, approval gates, artifact storage, traceability, logs, reports, and correction loops. The first implementation should focus on a working prototype foundation, not a perfect production system.

**Core Backend Principle**

Every agent output must be stored with project_id, artifact ID, status, approval state, created_by, timestamps, and traceability links. This prevents missing artifacts during integration.

- Use Python FastAPI as the backend framework.
- Use MongoDB as the Shared Knowledge Base storage layer.
- Use Pydantic schemas for request and response validation.
- Use a modular structure: router -> service -> repository -> database.
- Keep orchestrator logic centralized; do not duplicate workflow logic inside each agent.

# 2. Recommended Backend Technology Stack

| Area | Recommended Technology | Reason |
|---|---|---|
| Backend framework | FastAPI | Fast, simple, Python-based, suitable for AI workflows and REST APIs. |
| Validation | Pydantic | Keeps request and response structures consistent. |
| Database | MongoDB | Flexible JSON-style storage for requirements, designs, tests, logs, reports, and traceability. |
| Database driver | Motor or PyMongo | Motor for async implementation; PyMongo is acceptable for simple prototype. |
| Environment config | python-dotenv / Pydantic Settings | Keeps DB URL and app settings outside source code. |
| Testing | Pytest + HTTPX | Supports API-level backend tests. |
| Documentation | FastAPI OpenAPI docs | Automatically exposes API documentation for team testing. |

# 3. Backend Folder Structure

```
backend/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   ├── constants.py
│   │   └── errors.py
│   ├── database/
│   │   ├── mongodb.py
│   │   └── collections.py
│   ├── common/
│   │   ├── response.py
│   │   ├── status.py
│   │   ├── id_generator.py
│   │   └── traceability.py
```

```
│   ├── modules/
│   │   ├── projects/
│   │   ├── approvals/
│   │   ├── orchestrator/
│   │   ├── requirements/
│   │   ├── design/
│   │   ├── testing/
│   │   ├── implementation/
│   │   └── reports/
│   └── schemas/
├── tests/
├── .env.example
├── requirements.txt
└── README.md
```

| Folder | Owner | Rule |
|---|---|---|
| app/core | Foundation team | Global settings, constants, and shared errors only. |
| app/database | Foundation team | MongoDB connection and collection names only. |
| app/common | Foundation team | Shared response, status, ID, and traceability utilities. |
| app/modules/requirements | Requirement agent member | Only Requirement Gathering and Analysis logic. |
| app/modules/design | Design agent member | Only Project Management and Design logic. |
| app/modules/testing | Testing agent member | Pre-code and post-code test/security logic. |
| app/modules/implementation | Implementation agent member | Prototype generation, Git, logs, and self-fix flow. |
| app/modules/orchestrator | Leader/foundation team | Workflow, approval gates, routing, retries, and state. |

## 4. Standard Module Pattern

Every backend module should follow the same internal structure. This keeps coding consistent and makes AI-generated code easier to review.

```
app/modules/<module_name>/
├── router.py       # API endpoints only
├── service.py      # Business logic and workflow rules
├── repository.py   # Database read/write operations
├── schemas.py      # Module-specific request/response schemas
└── sample_data.py  # Mock/demo outputs for prototype testing
```

| Layer | Allowed Responsibility | Not Allowed |
|---|---|---|
| router.py | Define endpoints, call service, return response. | No business logic, no direct database code. |
| service.py | Validate workflow conditions, generate artifacts, call repository. | No raw HTTP response formatting. |
| repository.py | Database queries and updates only. | No UI logic, no workflow decisions. |
| schemas.py | Pydantic models for input/output. | No database queries, no generation logic. |
| sample_data.py | Temporary mock outputs for MVP demo. | Do not hardcode final production behavior. |

## 5. Common Status, Approval, and ID Rules

- All modules must use the same status values; do not invent component-specific statuses.
- Approval gates must use H1, H2, H3, H4, and H5 consistently.
- Artifact IDs must be human-readable because they are used in reports and traceability.

| Type | Allowed Values / Format |
|---|---|
| Workflow status | PENDING, IN_PROGRESS, WAITING_FOR_APPROVAL, APPROVED, REJECTED, FAILED, COMPLETED |
| Approval gates | H1_REQUIREMENTS, H2_DESIGN, H3_PRE_CODE_TESTS, H4_IMPLEMENTATION, H5_FINAL_VALIDATION |
| Artifact IDs | REQ-001, US-001, AC-001, DES-001, API-001, DB-001, TASK-001, TEST-001, CODE-001, BUG-001, SEC-001, TRACE-001 |
| Failure types | REQUIREMENT_ISSUE, DESIGN_ISSUE, BUILD_FAILURE, TEST_FAILURE, SECURITY_ISSUE |

## 6. API Naming and Response Rules

APIs must be predictable and grouped by project. Every important artifact belongs to a project, so most endpoints should include project_id in the URL.

```
Base pattern:
/api/projects/{project_id}/<module>/<action>

Examples:
POST /api/projects/{project_id}/requirements/process
GET  /api/projects/{project_id}/requirements
POST /api/projects/{project_id}/design/generate
POST /api/projects/{project_id}/testing/pre-code/generate
POST /api/projects/{project_id}/implementation/generate
GET  /api/projects/{project_id}/traceability
```

### 6.1 Standard Success Response

```
{
  "success": true,
  "message": "Requirements generated successfully",
  "data": { ... },
  "workflow_status": "H1_WAITING_APPROVAL",
  "traceability_updated": true
}
```

### 6.2 Standard Error Response

```
{
  "success": false,
  "error_code": "PROJECT_NOT_FOUND",
  "message": "Project was not found for the given project_id",
  "details": null
}
```

| Rule | Explanation |
|---|---|

| | |
|---|---|
| Use nouns in URLs | Use /requirements, /design, /testing, /implementation. |
| Use actions only when needed | Use /process, /generate, /run, /approve for workflow actions. |
| Return structured errors | Frontend should never parse plain text errors. |
| Include workflow status | Frontend workflow stepper depends on backend state. |
| Log important actions | Every major action should create workflow_events record. |

## 7. MongoDB Collection Plan

| Collection | Purpose | Written By |
|---|---|---|
| projects | Project profile, domain, owner, current stage, global status. | Foundation / Orchestrator |
| client_inputs | Initial client idea and uploaded input metadata. | Requirement agent |
| requirements | FR/NFR, user stories, acceptance criteria, REQ IDs. | Requirement agent |
| design_artifacts | UML, API plan, DB schema, UI flow, wireframe links. | Design agent |
| backlog_items | Epics, stories, tasks, subtasks, bugs, sprint items. | Design agent |
| test_cases | Pre-code and executable test specifications. | Testing agent |
| implementation_artifacts | Code module metadata, generated file tree, Git metadata. | Implementation agent |
| execution_logs | Build logs, runtime logs, self-fix logs, Docker run logs. | Implementation agent |
| test_results | Unit/API/integration/UI/regression results. | Testing agent |
| security_findings | Static/dependency/secret/API security findings. | Testing agent |
| bug_reports | Failed tests, defects, assigned fix status. | Testing + Design agents |
| approvals | Human approval decisions for H1-H5. | Orchestrator |
| traceability_links | Links between requirement, design, code, tests, bugs, security. | All agents |
| workflow_events | Audit trail of stage changes and handovers. | Orchestrator |

## 8. Required Base Fields for Every Artifact

```
{
  "project_id": "string",
  "artifact_id": "REQ-001",
  "artifact_type": "REQUIREMENT",
  "title": "string",
  "status": "PENDING | IN_PROGRESS | APPROVED | FAILED | COMPLETED",
  "approval_gate": "H1_REQUIREMENTS",
  "created_by_agent": "Requirement Gathering and Analysis Agent",
  "created_at": "ISO datetime",
  "updated_at": "ISO datetime",
  "version": 1,
  "traceability_refs": ["TRACE-001"]
}
```

**Backend Rule**

No component should save important data without project_id and artifact_id. Missing IDs will break final traceability

reports.

## 9. Orchestrator Integration Rules

The Orchestrator is the workflow controller. Individual agents should not decide the full system sequence by themselves. They complete their assigned work and notify the Orchestrator to move the workflow forward or route failures.

| Event | Backend Action |
|---|---|
| Agent output generated | Save artifact, create traceability links, set stage to WAITING_FOR_APPROVAL. |
| Human approves gate | Create approval record, update workflow stage, allow next agent. |
| Human rejects gate | Create approval record, return task to relevant agent. |
| Build/runtime failure | Route to Implementation and Git Automation Agent for limited self-fix. |
| Test failure | Route to Project Management and Design Agent for bug tracking and Implementation Agent for fix. |
| Security issue | Route to Project Management and Design Agent for issue tracking and Implementation Agent for fix. |

Recommended workflow stages:
PROJECT_CREATED
REQUIREMENTS_IN_PROGRESS
H1_WAITING_APPROVAL
DESIGN_IN_PROGRESS
H2_WAITING_APPROVAL
TEST_PLANNING_IN_PROGRESS
H3_WAITING_APPROVAL
IMPLEMENTATION_IN_PROGRESS
H4_WAITING_APPROVAL
VALIDATION_IN_PROGRESS
H5_WAITING_APPROVAL
COMPLETED

## 10. Backend Responsibilities by Component

| Component | Main Backend Outputs |
|---|---|
| Requirement Gathering and Analysis Agent | client_inputs, requirements, requirement quality report, H1 approval readiness, REQ/US/AC IDs. |
| Project Management and Design Agent | design_artifacts, backlog_items, API/DB/UI plans, requirement-to-design links, H2 approval readiness. |
| Testing, Validation and Security Agent - Pre-Code | test_cases, security test cases, requirement-to-test links, H3 approval readiness. |
| Implementation and Git Automation Agent | implementation_artifacts, generated module metadata, Git metadata, execution_logs, H4 approval readiness. |
| Testing, Validation and Security Agent - Post-Code | test_results, bug_reports, security_findings, validation report, H5 approval readiness. |
| Central Orchestrator | workflow_events, approvals, failure routing, retry counts, state transitions. |
| Shared Knowledge Base | MongoDB collections holding all approved artifacts and traceability evidence. |

## 11. Traceability Rules

Traceability is a research contribution of AgileFlow AI. The backend must preserve links between every major artifact.

Traceability path:
Requirement -> User Story -> Design Artifact -> API / DB / UI -> Test Case -> Code Module -> Bug / Security Finding -> Final Status

| Traceability Link Field | Example |
|---|---|
| source_type | REQUIREMENT |
| source_id | REQ-001 |
| target_type | TEST_CASE |
| target_id | TEST-001 |
| relationship | validated_by |
| status | ACTIVE |

- Agent 1 creates REQ, US, and AC IDs.
- Agent 2 links REQ IDs to Jira items, UML/API/DB/UI artifacts.
- Agent 4 links REQ/AC IDs to TEST IDs.
- Agent 3 links CODE IDs to REQ, API, DB, UI, and TEST IDs.
- Final validation links BUG and SEC IDs back to related requirements and code modules.

## 12. Initial API Contract for Backend Foundation

| Module | Endpoint | Purpose |
|---|---|---|
| Projects | POST /api/projects | Create a new project. |
| Projects | GET /api/projects | List projects. |
| Projects | GET /api/projects/{project_id} | Get project details. |
| Workflow | GET /api/projects/{project_id}/workflow | Get current workflow stage and approvals. |
| Requirements | POST /api/projects/{project_id}/requirements/process | Generate requirement package. |
| Requirements | GET /api/projects/{project_id}/requirements | View requirement artifacts. |
| Design | POST /api/projects/{project_id}/design/generate | Generate planning and design artifacts. |
| Testing | POST /api/projects/{project_id}/testing/pre-code/generate | Generate pre-code test package. |
| Implementation | POST /api/projects/{project_id}/implementation/generate | Generate prototype/code metadata. |
| Validation | POST /api/projects/{project_id}/validation/run | Run post-code validation and security report. |
| Approvals | POST /api/projects/{project_id}/approvals/{gate} | Approve/reject/request changes. |
| Traceability | GET /api/projects/{project_id}/traceability | Get traceability matrix. |

## 13. Error Handling and Logging Rules

| Error Code | When to Use |
|---|---|
| PROJECT_NOT_FOUND | No project exists for the provided project_id. |
| INVALID_WORKFLOW_STAGE | Action is not allowed in the current workflow stage. |
| APPROVAL_REQUIRED | Next agent cannot start before required human approval. |
| ARTIFACT_NOT_FOUND | Required requirement/design/test/code artifact is missing. |

| | |
|---|---|
| VALIDATION_ERROR | Request body or artifact structure is invalid. |
| GENERATION_FAILED | Agent output generation failed. |
| RETRY_LIMIT_EXCEEDED | Self-fix or correction loop exceeded allowed retry count. |

- Do not return raw Python exceptions to the frontend.
- Every failure that affects workflow should create a workflow_events record.
- Logs should be stored in execution_logs or workflow_events depending on context.

## 14. AI Coding Agent Safety Rules for Backend Work

**Required Rule**

When using an AI coding agent, always specify allowed files, do-not-modify files, current branch, API contract section, and expected output.

| Do | Do Not |
|---|---|
| Use existing schemas and collection names. | Create duplicate Project, Requirement, TestCase, or Approval models. |
| Work inside assigned module folder. | Edit another member's module without approval. |
| Update API_CONTRACT if a route changes. | Rename endpoints silently. |
| Run backend after AI changes. | Trust generated code without testing. |
| Return a changed-files summary. | Accept large unrelated edits. |

```
Backend AI prompt safety block:
Component: <module name>
Branch: <feature branch>
Allowed files/folders: <list>
Do not modify: app/core, app/database, other agent modules, shared schemas unless requested
Follow: API_CONTRACT.md, DATA_SCHEMAS.md, Implementation Engineering Handbook
Task: <specific task>
Output required: changed files summary + run/test instructions
```

## 15. Backend Testing and Acceptance Checklist

- Backend starts successfully with uvicorn.
- Swagger/OpenAPI page loads without errors.
- Project create/list/detail APIs work.
- Each agent API can return demo/mock output for the MVP.
- Approval gate APIs update workflow state correctly.
- Traceability API returns at least requirement-to-test/design/code links.
- Invalid project_id returns structured PROJECT_NOT_FOUND response.
- Invalid workflow action returns structured INVALID_WORKFLOW_STAGE response.
- No hardcoded database credentials in source code.
- All routes are grouped with proper FastAPI tags.

| Before Pull Request | Required Evidence |
|---|---|
| Code runs | Screenshot or terminal log showing uvicorn running. |
| API tested | Swagger/Postman/curl result for changed endpoints. |
| No unrelated edits | PR changed files only match assigned task. |
| Docs updated | API/data schema docs updated if contract changed. |

| | |
|---|---|
| Traceability preserved | New artifact IDs and traceability links saved where needed. |

## 16. Recommended Backend Start Order

| Order | Task | Owner |
|---|---|---|
| 1 | Create backend app structure and requirements.txt | Foundation team |
| 2 | Create config.py and .env.example | Foundation team |
| 3 | Create MongoDB connection and collections.py | Foundation team |
| 4 | Create common response, status, ID generator utilities | Foundation team |
| 5 | Create projects module | Foundation team |
| 6 | Create approvals module | Foundation team |
| 7 | Create orchestrator workflow module | Leader/foundation team |
| 8 | Create placeholder module routes for all agents | All members |
| 9 | Implement mock outputs for Agent 1 -> Agent 2 -> Agent 4 -> Agent 3 -> Agent 4 | Component owners |
| 10 | Connect final report and traceability APIs | Reports/orchestrator owner |

## 17. Simple Backend Rule Summary

**Backend Success Formula**

Common schemas + common responses + centralized workflow + shared knowledge base + traceability links = clean integration.

- Keep APIs project-based and predictable.
- Do not duplicate schemas, statuses, or ID formats.
- Do not put workflow decisions inside every agent.
- Always save outputs to MongoDB with project_id and artifact_id.
- Always update workflow_events after major actions.
- Always create traceability links for important artifacts.
- Use mock outputs first, then replace with real AI/tool integrations step by step.

**End of File 11 — Backend Implementation Guide**